

# Lab 10 – SQL In-Built Functions for Mathematical and String Operations

## Objective

The objective of this lab is to understand and use SQL Server built-in functions for:

- Mathematical calculations
- String manipulation
- Data type conversion
- Applying functions to table columns
- Filtering rows using functions

---

## PART 1: MATHEMATICAL FUNCTIONS

### 1. Arithmetic Operators

SQL can directly perform mathematical calculations.

#### Syntax

```
SELECT number1 + number2;  
SELECT number1 - number2;  
SELECT number1 * number2;  
SELECT number1 / number2;  
SELECT number1 % number2;
```

#### Operators

| Operator     | Meaning             |
|--------------|---------------------|
| <div>+</div> | Addition            |
| <div>-</div> | Subtraction         |
| <div>*</div> | Multiplication      |
| <div>/</div> | Division            |
| <div>%</div> | Remainder (Modulus) |

## Example

```
SELECT 10 * 4 AS RESULT;  
SELECT 10 % 3 AS REMAINDER;
```

---

## 2. ABS()

`ABS()` returns the absolute value of a number.

Absolute value means the number without its negative sign.

### Syntax

```
ABS(number)
```

## Example

```
SELECT ABS(-40);  
SELECT ABS(40);
```

Both return:

```
40
```

---

## 3. CEILING()

`CEILING()` returns the smallest integer greater than or equal to the given number.

### Syntax

```
CEILING(number)
```

## Examples

```
SELECT CEILING(12.3);  
SELECT CEILING(-12.3);
```

Results:

```
13  
-12
```

### Important

CEILING moves toward the greater value.

---

## 4. FLOOR()

`FLOOR()` returns the largest integer smaller than or equal to the given number.

### Syntax

```
FLOOR(number)
```

### Examples

```
SELECT FLOOR(12.8);  
SELECT FLOOR(-12.3);
```

Results:

```
12  
-13
```

### Important

FLOOR moves toward the smaller value.

---

## 5. Modulus Operator (%)

The `%` operator returns the remainder after division.

### Syntax

```
number1 % number2
```

### Example

```
SELECT 17 % 5;
```

Result:

```
2
```

---

## 6. POWER()

`POWER()` raises one number to the power of another number.

### Syntax

```
POWER(number , power )
```

### Example

```
SELECT POWER(5, 3);
```

Meaning:

```
5 × 5 × 5
```

---

## 7. SQRT()

`SQRT()` returns the square root of a number.

### Syntax

```
SQRT(number)
```

### Example

```
SELECT SQRT(36);
```

Result:

6

---

## 8. SQUARE()

`SQUARE()` returns the square of a number.

### Syntax

```
SQUARE(number)
```

### Example

```
SELECT SQUARE(8);
```

Meaning:

$8 \times 8$

---

## 9. PI()

`PI()` returns the mathematical value of  $\pi$ .

### Syntax

```
PI()
```

### Example

```
SELECT PI();
```

---

## 10. ROUND()

`ROUND()` rounds a number to the specified number of decimal places.

### Syntax

```
ROUND(number, decimal_position)
```

### Example

```
SELECT ROUND(145.678, 2);
```

Result:

```
145.680
```

### Decimal Position

2 → Round to 2 digits after decimal point

1 → Round to 1 digit after decimal point

0 → Round to nearest integer

-1 → Round to nearest 10

-2 → Round to nearest 100

### Example

```
SELECT ROUND(167.45, -2);
```

Result:

```
200.00
```

## 11. EXP()

`EXP()` returns the value of `e` raised to the specified power.

### Syntax

```
EXP(number)
```

### Example

```
SELECT EXP(4);
```

Meaning:

```
e4
```

---

## 12. LOG()

`LOG()` returns the natural logarithm of a number.

Natural logarithm uses base `e`.

### Syntax

```
LOG(number)
```

### Example

```
SELECT LOG(20);
```

---

## 13. LOG10()

`LOG10()` returns the logarithm using base 10.

## Syntax

```
LOG10(number)
```

## Example

```
SELECT LOG10(1000);
```

Result:

```
3
```

Because:

```
103 = 1000
```

---

## 14. SIN(), COS() and TAN()

These functions calculate trigonometric values.

## Syntax

```
SIN(number)  
COS(number)  
TAN(number)
```

## Example

```
SELECT SIN(1.5);  
SELECT COS(1.5);  
SELECT TAN(1.5);
```

## Important

SQL Server trigonometric functions use radians.

---

## 15. SIGN()

`SIGN()` checks whether a number is positive, negative, or zero.

### Syntax

```
SIGN(number)
```

### Return Values

| Number          | Result |
|-----------------|--------|
| Negative number | -1     |
| Zero            | 0      |
| Positive number | 1      |

### Example

```
SELECT SIGN(-100);  
SELECT SIGN(0);  
SELECT SIGN(100);
```

## 16. RAND()

`RAND()` generates a pseudo-random floating-point number.

### Syntax

```
RAND()
```

### Example

```
SELECT RAND();
```

The value is generally:

```
0 <= value < 1
```

## PART 2: STRING FUNCTIONS

### 1. LEN()

`LEN()` returns the number of characters in a string.

#### Syntax

```
LEN(string)
```

#### Example

```
SELECT LEN('DATABASE');
```

Result:

```
8
```

#### Important Behavior

`LEN()` ignores trailing spaces.

```
SELECT LEN('SQL ');
```

Result:

```
3
```

Leading spaces are counted.

```
SELECT LEN(' SQL');
```

Result:

```
6
```

`LEN(NULL)` returns `NULL`.

```
SELECT LEN(NULL);
```

An empty string has length `0`.

```
SELECT LEN('');
```

---

## 2. LOWER()

`LOWER()` converts characters to lowercase.

### Syntax

```
LOWER(string)
```

### Example

```
SELECT LOWER('DATABASE');
```

Result:

```
database
```

---

## 3. UPPER()

`UPPER()` converts characters to uppercase.

### Syntax

```
UPPER(string)
```

### Example

```
SELECT UPPER('database');
```

Result:

DATABASE

---

## 4. LEFT()

`LEFT()` extracts characters from the beginning of a string.

### Syntax

```
LEFT(string, number_of_characters)
```

### Example

```
SELECT LEFT('DATABASE', 4);
```

Result:

DATA

---

## 5. RIGHT()

`RIGHT()` extracts characters from the end of a string.

### Syntax

```
RIGHT(string, number_of_characters)
```

### Example

```
SELECT RIGHT('DATABASE', 4);
```

Result:

BASE

---

## 6. SUBSTRING()

`SUBSTRING()` extracts characters from a specific position.

### Syntax

```
SUBSTRING(string, start_position, length)
```

### Example

```
SELECT SUBSTRING('DATABASE SYSTEM', 3, 5);
```

### Important

SQL Server string positions start from `1`.

```
D A T A B A S E  
1 2 3 4 5 6 7 8
```

Therefore:

```
SUBSTRING('DATABASE', 3, 4)
```

starts from character `T` and extracts `4` characters.

---

## 7. REPLACE()

`REPLACE()` replaces all occurrences of one substring with another substring.

### Syntax

```
REPLACE(original_string, old_string, new_string)
```

### Example

```
SELECT REPLACE('DATABASE', 'A', '@');
```

Result:

D@T@B@SE

## Important

All matching occurrences are replaced.

---

## 8. Nested REPLACE()

Multiple `REPLACE()` functions can be placed inside each other.

### Syntax Pattern

```
REPLACE(  
    REPLACE(  
        column_name,  
        'old_value1',  
        'new_value'  
    ),  
    'old_value2',  
    'new_value'  
)
```

### Example

```
SELECT REPLACE(  
    REPLACE('DATABASE', 'A', '*'),  
    'E',  
    '*'  
);
```

### Execution Order

SQL evaluates the inner function first.

Original String

↓

Inner REPLACE()

↓

Result passed to outer REPLACE()

↓

Final Result

This pattern can be extended when multiple characters must be replaced.

---

## 9. ASCII()

`ASCII()` returns the ASCII code of the first character.

### Syntax

```
ASCII(character)
```

### Example

```
SELECT ASCII('B');
```

Result:

```
66
```

### Common ASCII Ranges

```
A - Z → 65 - 90
```

```
a - z → 97 - 122
```

```
0 - 9 → 48 - 57
```

---

## 10. CHAR()

`CHAR()` converts an ASCII number into its corresponding character.

## Syntax

```
CHAR(ascii_number)
```

## Example

```
SELECT CHAR(66);
```

Result:

```
B
```

## Relationship

```
Character → ASCII() → Number
```

```
Number → CHAR() → Character
```

---

## 11. LTRIM()

`LTRIM()` removes spaces from the left side of a string.

## Syntax

```
LTRIM(string)
```

## Example

```
SELECT LTRIM('  DATABASE ');
```

Result conceptually:

```
'DATABASE  '
```

Right-side spaces remain.

---

## 12. RTRIM()

`RTRIM()` removes spaces from the right side of a string.

### Syntax

```
RTRIM(string)
```

### Example

```
SELECT RTRIM('   DATABASE  ');
```

Result conceptually:

```
'   DATABASE'
```

Left-side spaces remain.

---

## DATA TYPE CONVERSION

## 13. CAST()

`CAST()` converts a value from one data type to another data type.

### Syntax

```
CAST(value AS datatype)
```

### Example

```
SELECT CAST('789.45' AS DECIMAL(7,2));
```

### Understanding DECIMAL(p, s)

```
DECIMAL(7,2)
```

7 → Maximum total number of digits

2 → Maximum digits after decimal point

Therefore:

12345.67

can fit inside:

DECIMAL(7,2)

---

## 14. CONVERT()

`CONVERT()` also converts a value from one data type to another.

### Syntax

```
CONVERT(datatype, value)
```

### Example

```
SELECT CONVERT(DECIMAL(7,2), '789.45');
```

### CAST vs CONVERT

```
CAST(value AS datatype)
```

```
CONVERT(datatype, value)
```

The main difference in basic lab questions is the syntax.

`CAST()` is standard SQL syntax.

`CONVERT()` is commonly associated with SQL Server and also supports style parameters for some conversions.

---

## 15. Converting Decimal or Float to INT

### Syntax

```
CAST(decimal_value AS INT)

CONVERT(INT, decimal_value)
```

### Example

```
SELECT CAST(25.89 AS INT);
```

Result:

```
25
```

### Important

Converting a decimal value to `INT` removes the fractional part.

It does not perform normal mathematical rounding.

---

## 16. SPACE()

`SPACE()` generates a specified number of spaces.

### Syntax

```
SPACE(number)
```

### Example

```
SELECT SPACE(5) + 'DATABASE';
```

---

## 17. String Concatenation Using +

Strings can be joined using the `+` operator.

## Syntax

```
string1 + string2
```

## Example

```
SELECT 'SQL' + ' ' + 'SERVER';
```

Result:

```
SQL SERVER
```

## Important NULL Behavior

```
SELECT 'SQL' + NULL;
```

Normally returns:

```
NULL
```

Therefore, `+` can cause problems when a column contains `NULL`.

---

## 18. CONCAT()

`CONCAT()` combines multiple strings.

## Syntax

```
CONCAT(value1, value2, value3, ...)
```

## Example

```
SELECT CONCAT('SQL', ' ', 'SERVER');
```

## Important

`CONCAT()` automatically converts values to strings.

`CONCAT()` treats `NULL` as an empty string.

Example:

```
SELECT CONCAT('SQL', NULL, 'SERVER');
```

Result:

```
SQLSERVER
```

---

## 19. REVERSE()

`REVERSE()` reverses a string.

### Syntax

```
REVERSE(string)
```

### Example

```
SELECT REVERSE('DATABASE');
```

Result:

```
ESABATAD
```

---

## 20. REPLICATE()

`REPLICATE()` repeats a string a specified number of times.

### Syntax

```
REPLICATE(string, number_of_times)
```

## Example

```
SELECT REPLICATE('SQL', 4);
```

Result:

```
SQLSQLSQLSQL
```

---

## APPLYING FUNCTIONS TO TABLE COLUMNS

Built-in functions can be applied to individual values or table columns.

General syntax:

```
SELECT FUNCTION_NAME(column_name)
FROM table_name;
```

Example:

```
SELECT UPPER(column_name)
FROM table_name;
```

SQL applies the function separately to every selected row.

---

## Applying Multiple Functions

Multiple functions can be used in one `SELECT`.

### Pattern

```
SELECT
    FUNCTION1(column1),
    FUNCTION2(column2)
FROM table_name;
```

## Creating a Full Name

Two columns can be combined using `+`.

### Pattern

```
SELECT
    column1 + ' ' + column2 AS FULL_NAME
FROM table_name;
```

Or using `CONCAT()`:

```
SELECT
    CONCAT(column1, ' ', column2) AS FULL_NAME
FROM table_name;
```

---

## Extracting First N Characters

### Pattern

```
SELECT LEFT(column_name, N)
FROM table_name;
```

---

## Extracting Last N Characters

### Pattern

```
SELECT RIGHT(column_name, N)
FROM table_name;
```

---

## Finding Length of Every Column Value

### Pattern

```
SELECT
    column_name,
```

```
LEN(column_name) AS COLUMN_LENGTH  
FROM table_name;
```

## Replacing Characters in a Column

### Pattern

```
SELECT  
    REPLACE(column_name, 'old_character', 'new_character')  
FROM table_name;
```

## IMPORTANT PART C QUERY PATTERNS

### 1. Removing the First and Last Character

Suppose a string contains:

DATABASE

Length:

8

We want:

ATABAS

The first character must be skipped.

Therefore, start position:

2

The first and last characters must be removed.

Therefore, required length:

Total Length - 2

### General Pattern

```
SUBSTRING(column_name, 2, LEN(column_name) - 2)
```

## 2. Replacing Multiple Characters

`REPLACE()` handles one search string at a time.

When many characters must be replaced, use nested `REPLACE()`.

### General Pattern

```
REPLACE(  
    REPLACE(  
        REPLACE(  
            column_name,  
            'character1',  
            'replacement'  
        ),  
        'character2',  
        'replacement'  
    ),  
    'character3',  
    'replacement'  
)
```

### Execution

Original Value

↓

Replace Character 1

↓

Replace Character 2

↓

Replace Character 3

↓

Final Value

## Important

Depending on the database collation, character comparisons may be case-insensitive or case-sensitive.

If the task requires handling uppercase and lowercase characters explicitly, both forms may need to be considered.

## 3. Filtering Using Combined Length

Functions can be used inside the `WHERE` clause.

### Pattern

```
SELECT *  
FROM table_name  
WHERE LEN(column1) + LEN(column2) > required_length;
```

### Execution Order

FROM table

↓

Read each row

↓

Calculate LEN(column1)

↓

Calculate LEN(column2)

↓

Add both lengths

↓

Check WHERE condition

↓

Return matching rows

## 4. Displaying Original and Modified Values

A useful pattern is displaying the original column and function result together.

### Pattern

```
SELECT
    column_name,
    FUNCTION_NAME(column_name) AS MODIFIED_VALUE
FROM table_name;
```

Example:

```
SELECT
    column_name,
    REVERSE(column_name) AS REVERSED_VALUE
FROM table_name;
```

## 5. Comparing the First Character of Two Columns

Extract the first character from both columns and compare them.

### Pattern

```
WHERE LEFT(column1, 1) = LEFT(column2, 1)
```

### Execution

Take Column 1 Value

↓

LEFT(column1, 1)

↓

First Character

Take Column 2 Value

↓

LEFT(column2, 1)

↓

First Character

Compare Both Characters

↓

TRUE → Return Row

FALSE → Ignore Row

---

## FUNCTION EXECUTION: INNER FUNCTION FIRST

When functions are nested, SQL evaluates the innermost function first.

Example pattern:

```
FUNCTION1(FUNCTION2(column_name))
```

Execution:

column\_name

↓

FUNCTION2()

↓

Result

↓

FUNCTION1()

↓

Final Result

Example:

```
SELECT UPPER(REVERSE(column_name))  
FROM table_name;
```

Execution:

Original Value

↓

REVERSE()

↓

UPPER()

↓

Final Result

---

## USING FUNCTIONS IN WHERE

Functions can be used in conditions.

### Pattern

```
SELECT *  
FROM table_name  
WHERE FUNCTION_NAME(column_name) condition;
```

Examples of useful patterns:

```
WHERE LEN(column_name) > value
```

```
WHERE LEFT(column_name, 1) = 'character'
```

```
WHERE RIGHT(column_name, 1) = 'character'
```

```
WHERE LOWER(column_name) = 'value'
```

```
WHERE FUNCTION1(column1) = FUNCTION2(column2)
```

## IMPORTANT DIFFERENCES TO REMEMBER

| Function / Operator | Purpose                                          |
|---------------------|--------------------------------------------------|
| ABS()               | Remove negative sign                             |
| CEILING()           | Smallest integer greater than or equal to number |
| FLOOR()             | Largest integer smaller than or equal to number  |
| %                   | Find remainder                                   |
| POWER()             | Raise number to a power                          |
| SQRT()              | Find square root                                 |
| SQUARE()            | Find square                                      |
| PI()                | Return $\pi$                                     |
| ROUND()             | Round a number                                   |
| EXP()               | Calculate e raised to a power                    |
| LOG()               | Natural logarithm                                |
| LOG10()             | Base-10 logarithm                                |
| SIN()               | Find sine                                        |
| COS()               | Find cosine                                      |
| TAN()               | Find tangent                                     |
| SIGN()              | Check positive, negative, or zero                |
| RAND()              | Generate pseudo-random number                    |

| Function / Operator      | Purpose                            |
|--------------------------|------------------------------------|
| <code>LEN()</code>       | Find string length                 |
| <code>LOWER()</code>     | Convert to lowercase               |
| <code>UPPER()</code>     | Convert to uppercase               |
| <code>LEFT()</code>      | Extract characters from left       |
| <code>RIGHT()</code>     | Extract characters from right      |
| <code>SUBSTRING()</code> | Extract characters from a position |
| <code>REPLACE()</code>   | Replace matching text              |
| <code>ASCII()</code>     | Character to ASCII number          |
| <code>CHAR()</code>      | ASCII number to character          |
| <code>LTRIM()</code>     | Remove leading spaces              |
| <code>RTRIM()</code>     | Remove trailing spaces             |
| <code>CAST()</code>      | Convert data type                  |
| <code>CONVERT()</code>   | Convert data type                  |
| <code>SPACE()</code>     | Generate spaces                    |
| <code>+</code>           | Concatenate strings in SQL Server  |
| <code>CONCAT()</code>    | Combine multiple values            |
| <code>REVERSE()</code>   | Reverse a string                   |
| <code>REPLICATE()</code> | Repeat a string                    |

## COMMON MISTAKES

### Mistake 1: Confusing CEILING and FLOOR

`CEILING` → Greater integer

`FLOOR` → Smaller integer

Be especially careful with negative numbers.

```
CEILING(-4.7) = -4
```

```
FLOOR(-4.7) = -5
```

---

## Mistake 2: Confusing ROUND with CAST to INT

```
ROUND(15.8, 0)
```

performs mathematical rounding.

```
CAST(15.8 AS INT)
```

removes the decimal portion.

These operations are not the same.

---

## Mistake 3: Wrong SUBSTRING Length

Remember:

```
SUBSTRING(string, start_position, length)
```

The third argument is the number of characters to extract.

It is not the ending position.

---

## Mistake 4: Forgetting SQL String Positions Start at 1

```
First Character → Position 1
```

```
Second Character → Position 2
```

```
Third Character → Position 3
```

## Mistake 5: Thinking REPLACE Changes Only One Character

`REPLACE()` changes all matching occurrences.

---

## Mistake 6: Wrong CAST Syntax

Correct:

```
CAST(value AS datatype)
```

Incorrect pattern:

```
CAST(datatype, value)
```

---

## Mistake 7: Wrong CONVERT Syntax

Correct:

```
CONVERT(datatype, value)
```

---

## Mistake 8: Forgetting LEN Ignores Trailing Spaces

```
LEN('SQL ')
```

returns:

```
3
```

not `6`.

---

## Mistake 9: Using + When Columns May Contain NULL

```
column1 + column2
```

can produce `NULL` when one value is `NULL`.

Using:

```
CONCAT(column1, column2)
```

is safer when `NULL` values are possible.

---

## Mistake 10: Forgetting INNER Functions Execute First

For:

```
FUNCTION1(FUNCTION2(value))
```

SQL executes:

```
FUNCTION2 → FUNCTION1
```

---

## QUICK LAB PROBLEM-SOLVING GUIDE

When reading a question, identify the required operation.

Absolute value

↓

`ABS()`

Next greater integer

↓

`CEILING()`

Previous smaller integer

↓  
FLOOR()

Remainder

↓  
%

Power

↓  
POWER()

Square root

↓  
SQRT()

Square

↓  
SQUARE()

Round number

↓  
ROUND()

String length

↓  
LEN()

Convert case

↓  
LOWER() / UPPER()

First characters

↓  
LEFT()

Last characters

↓  
RIGHT()

Characters from a specific position

↓

SUBSTRING()

Change characters or text

↓

REPLACE()

Character to ASCII number

↓

ASCII()

ASCII number to character

↓

CHAR()

Remove left spaces

↓

LTRIM()

Remove right spaces

↓

RTRIM()

Change data type

↓

CAST() / CONVERT()

Generate spaces

↓

SPACE()

Combine strings

↓

+ / CONCAT()

Reverse string

↓

REVERSE()

Repeat string

↓

REPLICATE()

Replace many different characters

↓

Nested REPLACE()

Filter based on calculated string length

↓

LEN() inside WHERE

Compare first characters

↓

LEFT(column1, 1) = LEFT(column2, 1)

---

## FINAL REVISION CHECKLIST

Before performing the lab, make sure you can write these syntaxes without looking:

ABS(number)

CEILING(number)

FLOOR(number)

number1 % number2

POWER(number, power)

SQRT(number)

SQUARE(number)

PI()

ROUND(number, decimal\_position)

EXP(number)

LOG(number)

LOG10(number)

SIN(number)

COS(number)

TAN(number)

SIGN(number)

RAND()

LEN(string)

LOWER(string)

UPPER(string)

LEFT(string, length)

RIGHT(string, length)

SUBSTRING(string, start\_position, length)

REPLACE(string, old\_value, new\_value)

ASCII(character)

CHAR(number)

LTRIM(string)

RTRIM(string)

CAST(value AS datatype)

CONVERT(datatype, value)

SPACE(number)

string1 + string2

```
CONCAT(value1, value2, ...)

REVERSE(string)

REPLICATE(string, number_of_times)
```

## Most Important Patterns for Table-Based Questions

```
SELECT FUNCTION(column_name)
FROM table_name;
```

```
SELECT
    column_name,
    FUNCTION(column_name) AS RESULT
FROM table_name;
```

```
SELECT *
FROM table_name
WHERE FUNCTION(column_name) condition;
```

```
SUBSTRING(column_name, 2, LEN(column_name) - 2)
```

```
REPLACE(REPLACE(column_name, 'x', '*'), 'y', '*')
```

```
WHERE LEN(column1) + LEN(column2) > value
```

```
WHERE LEFT(column1, 1) = LEFT(column2, 1)
```

## One Rule to Remember

First identify what the question wants, choose the correct function, place the value or column inside that function, and use `WHERE` only when rows need to be filtered.